

NGRX

Redux for Angular

Paul Mestrum / 21.11.24



delaware



delaware

ten industries



manufacturing



automotive



pharma



chemicals



retail



aerospace



utilities



food



healthcare



print & packaging

global footprint of delaware office locations

4,600 professionals

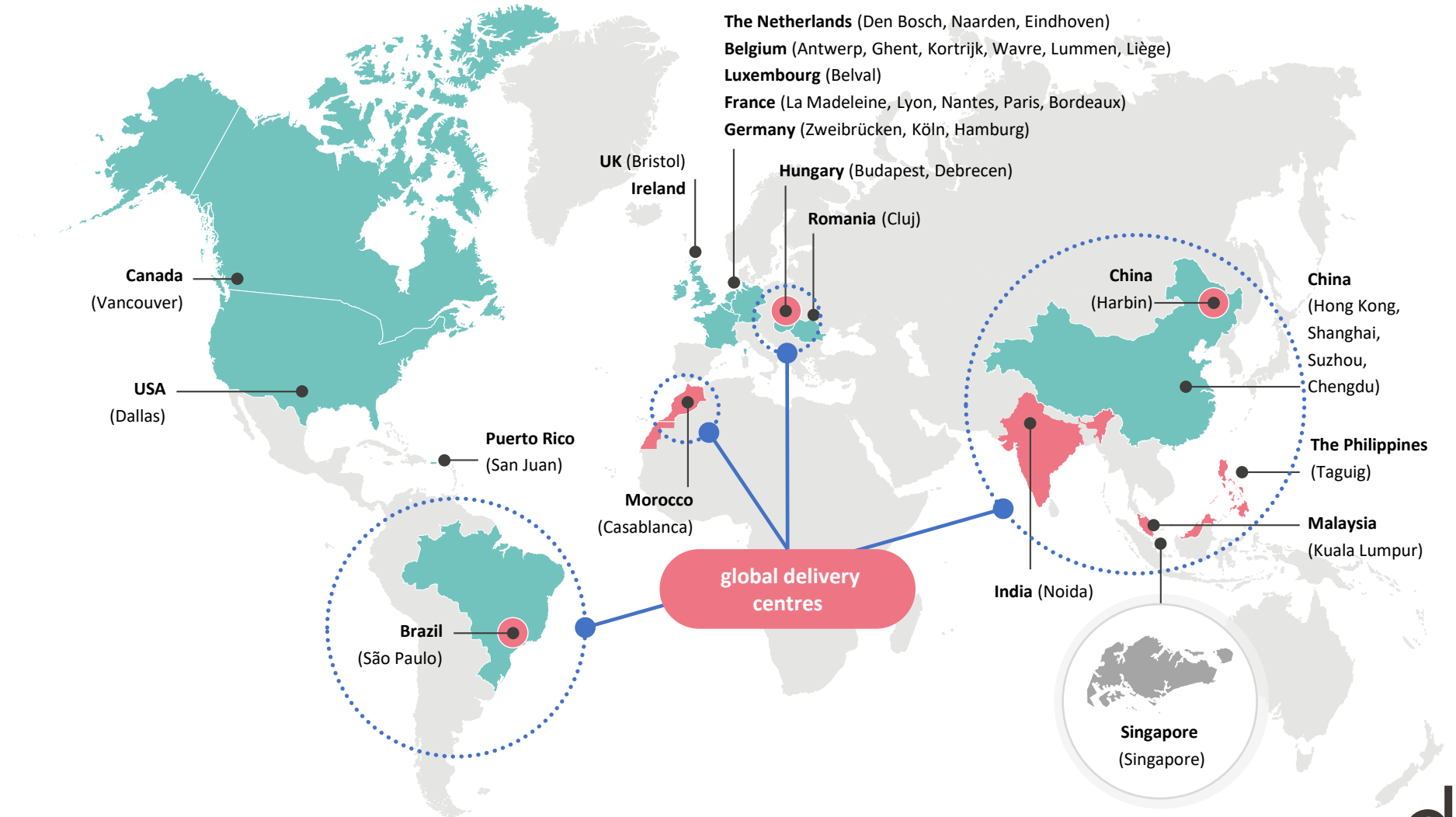
37 offices in
19 countries

€490 mil
revenue

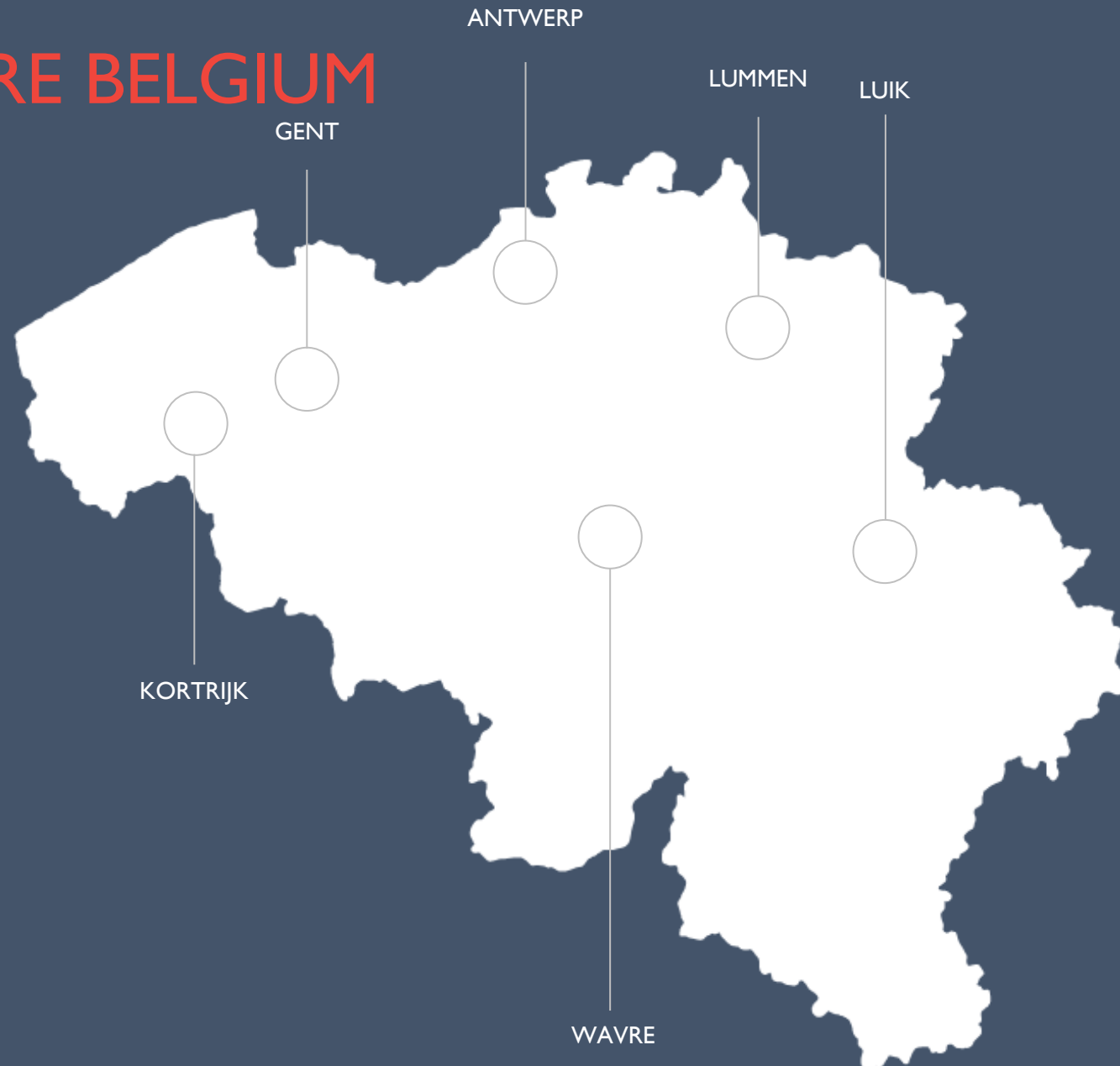
global delivery network



our partner ecosystem



DELAWARE BELGIUM





Mestrum, Paul

Senior Consultant
Mobile Web & IoT - Connect
Systems & People
Ghent - Belgium
☎ +32 474 63 71 84



- **Frontend development**

- Angular
- React
- Technologies
 - Ionic & cordova
 - PWA

- **Projects**

- Web applications
- xADP: mobile, desktop & web
- IoT
- Chats / assistants

Middleware development

- › Dot NET core – C#
- › Technologies
 - › Entity framework
 - › SQL & cosmos DB

AR / VR

- › Unity

Full stack

- › FE + MW

DevOps / Azure (& sometimes SAP cloud)



Starting your career at delaware?

Check out our [career website...](#)



...or get in touch with one of our [junior recruiter](#)
Isabel Bergsma Isabel.bergsma@delaware.pro



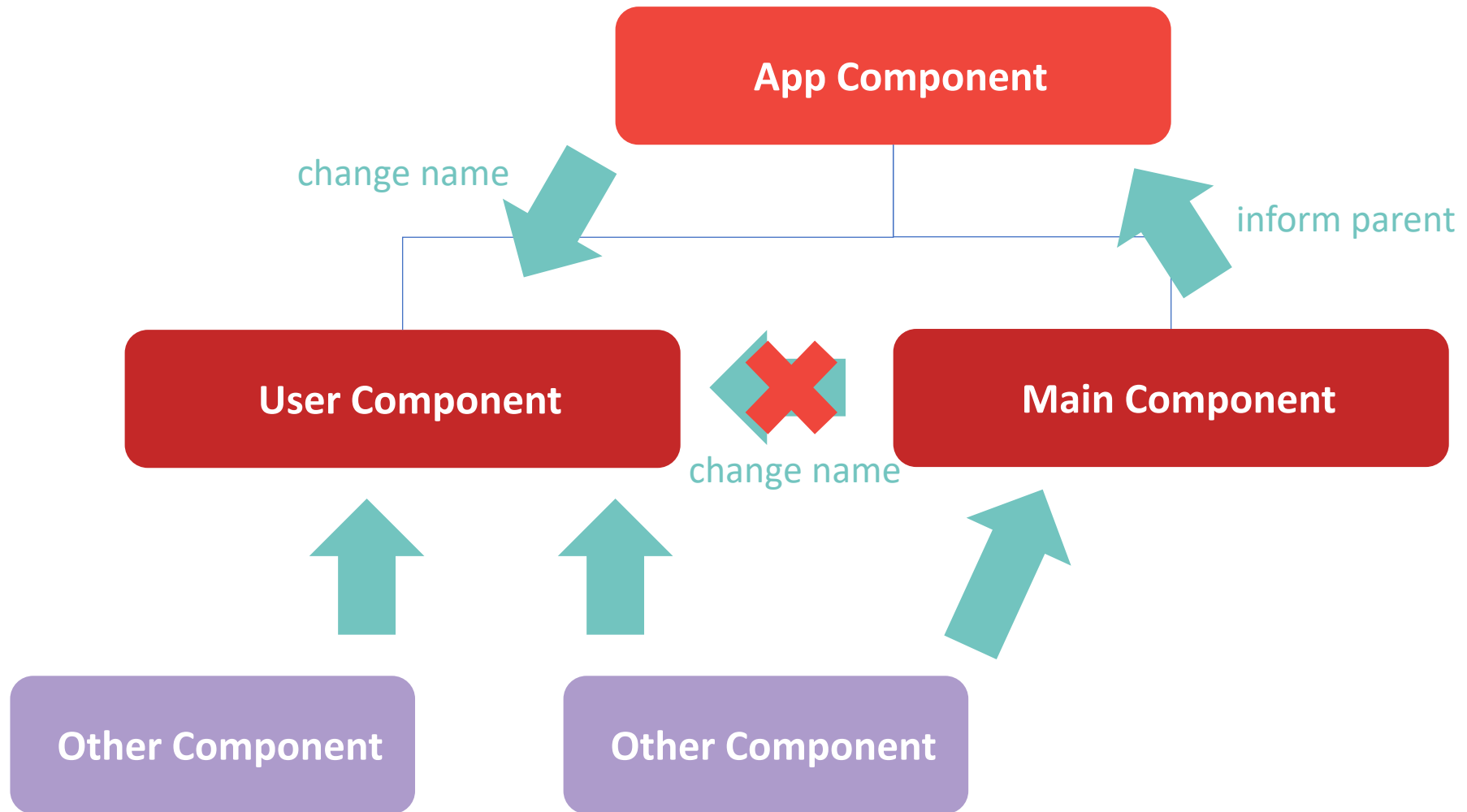
NGRX

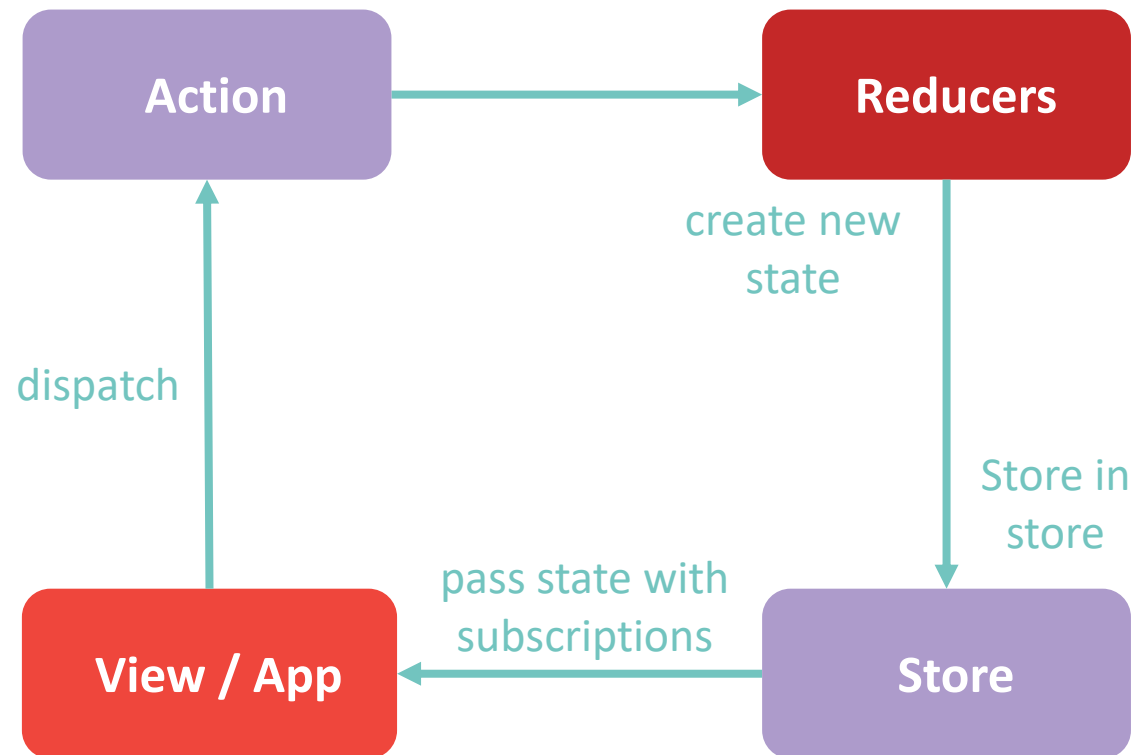
Redux for Angular

Paul Mestrum / 08.11.23



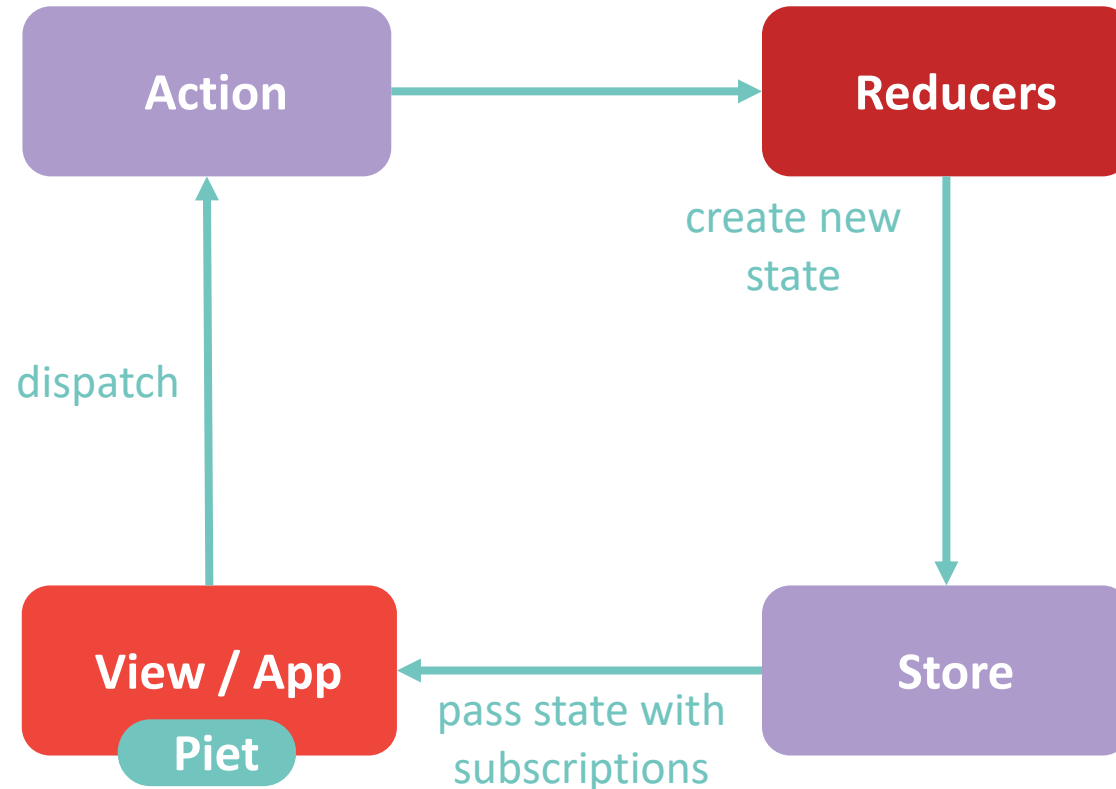
delaware





Action:

```
{  
  type: 'CHANGE_NAME'  
  payload 'Piet'  
}
```

**Selector:**

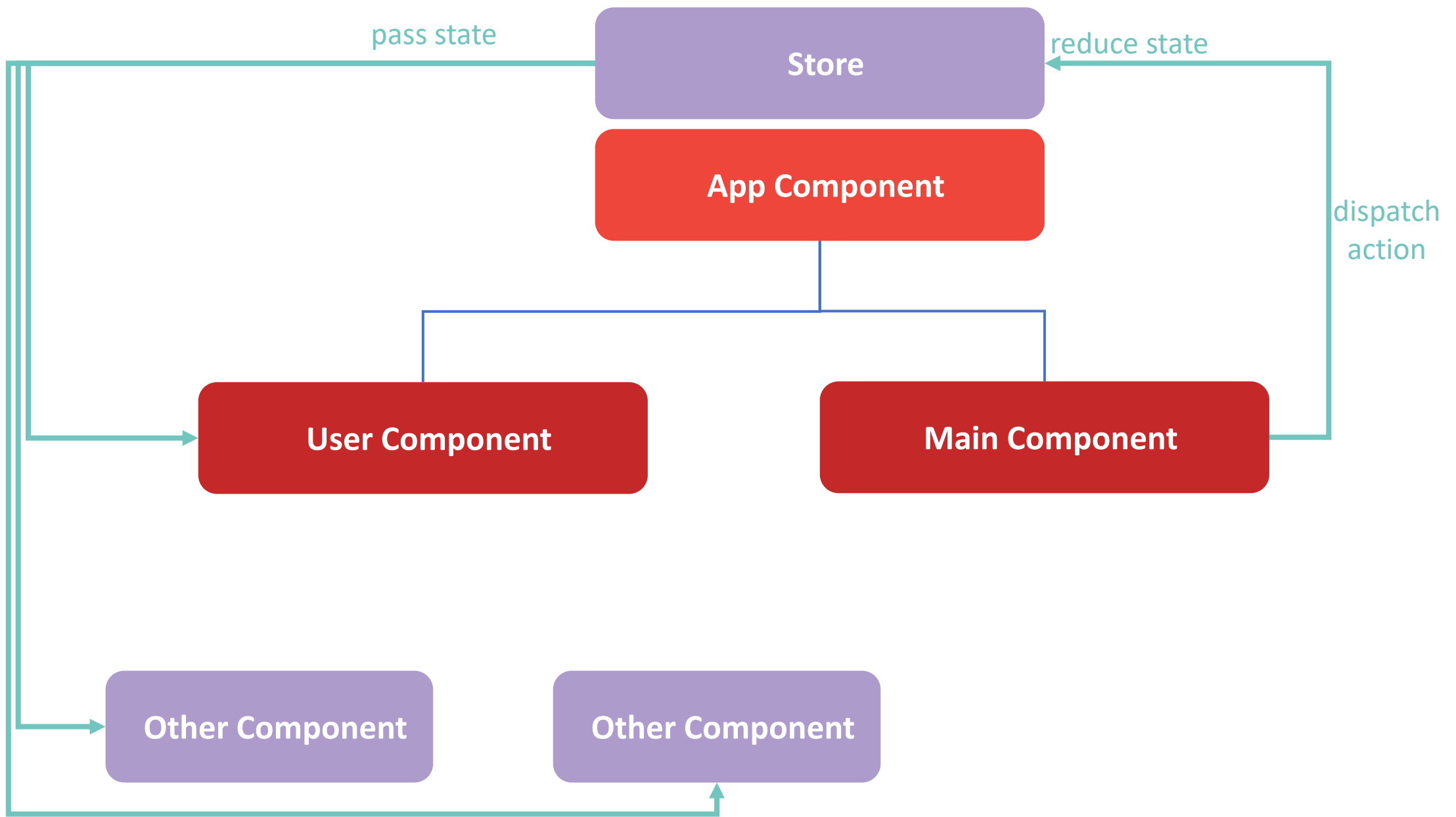
```
username$ = store.select(state => state.user?.name)
```

Reducer:

```
function reduce(state, action) {  
  switch (action.type) {  
    case 'CHANGE_NAME':  
      return {  
        ...state,  
        user: {name: action.payload}  
      };  
    default:  
      return state;  
  }  
}
```

Store:

```
{  
  user: {  
    name: "Piet"  
  }  
}
```





ngrx

=



redux Pattern

+



Angular

+



RxJS

Redux principles

- Single source of truth, called “The Store”
- State is read-only and can only be changed by dispatching actions
- Changes are made using pure functions called reducers

Actions

- Action = object with type and payload:

```
{  type: 'LOGIN',  
  payload: {user: 'johnny', pwd:  
'secret'}  
}
```

- Examples:
 - Adding items in the cart
 - Storing UI settings (eg active tab/page => breadcrumbs, activated tab, impact on menu)

Creating actions

- Using a class:

```
export class LoginAction extends Action {  
  type: 'LOGIN',  
  constructor(payload: {user: string, pwd: string}){};  
}  
  
const action = new LoginAction({user: 'x', pwd: 'x'});
```

- Using a create function:

```
export const loginAction = createAction('LOGIN',  
  props<{user: string, pwd: string}>());  
  
const action = loginAction({user: 'x', pwd: 'x'});
```

Reducers

Reducers

Reducer = function that manipulate the state based on incoming actions

```
function reducer(state, action): state;
```

Create a reducer function

Reducers

```
function reducer(state: State = INITIAL_STATE, action: Actions):  
State {  
  switch (action.type) {  
    case 'LOAD_USER':  
      return {  
        ...state,  
        users: [...state.users, action.payload]  
      }  
    return state;  
  }  
}
```

New way of creating a reducer function

```
const reducer = createReducer(INITIAL_STATE,  
  on(loadUserAction, (state, payload) => ({  
    ...state,  
    users: [...state.users, action.payload]  
  }));  
}
```

Reducers are pure functions

Reducers

- A pure function is a function which:
 - Given the same input, will always return the same output.
 - Produces no side effects.

```
function sum(a, b) {  
  const result = a +  
b;  
  return result;  
}
```

```
let c = 1;  
  
function sum(a, b) {  
  result = a + b + c;  
  return result;  
}
```

Selectors

- An observable that selects and maps the state

```
const username$  
  = this.store.select( state => {  
    return state.user.name || '- No name -';  
  } );
```

Using a create function:

```
const username$  
  = createSelector( state => state.user,  
    user => user.name || '-- no name --' );
```


Observables ?

- An RxJS Observable is a data stream that can emit multiple values over time, allowing you to handle asynchronous events and data in a structured way

```
store.select(state => state.user)
  .pipe(
    map(userState => userState.name),
    debounceTime(1000),
    distinctUntilChanged()
  ).subscribe(username => {
    ...
  });
```



Do you have to put everything in the state ?

Checklist: SHARI

Shared between many components and services

Hydrated (persisted) from (local/session/?)storage

Available when re-entering routes

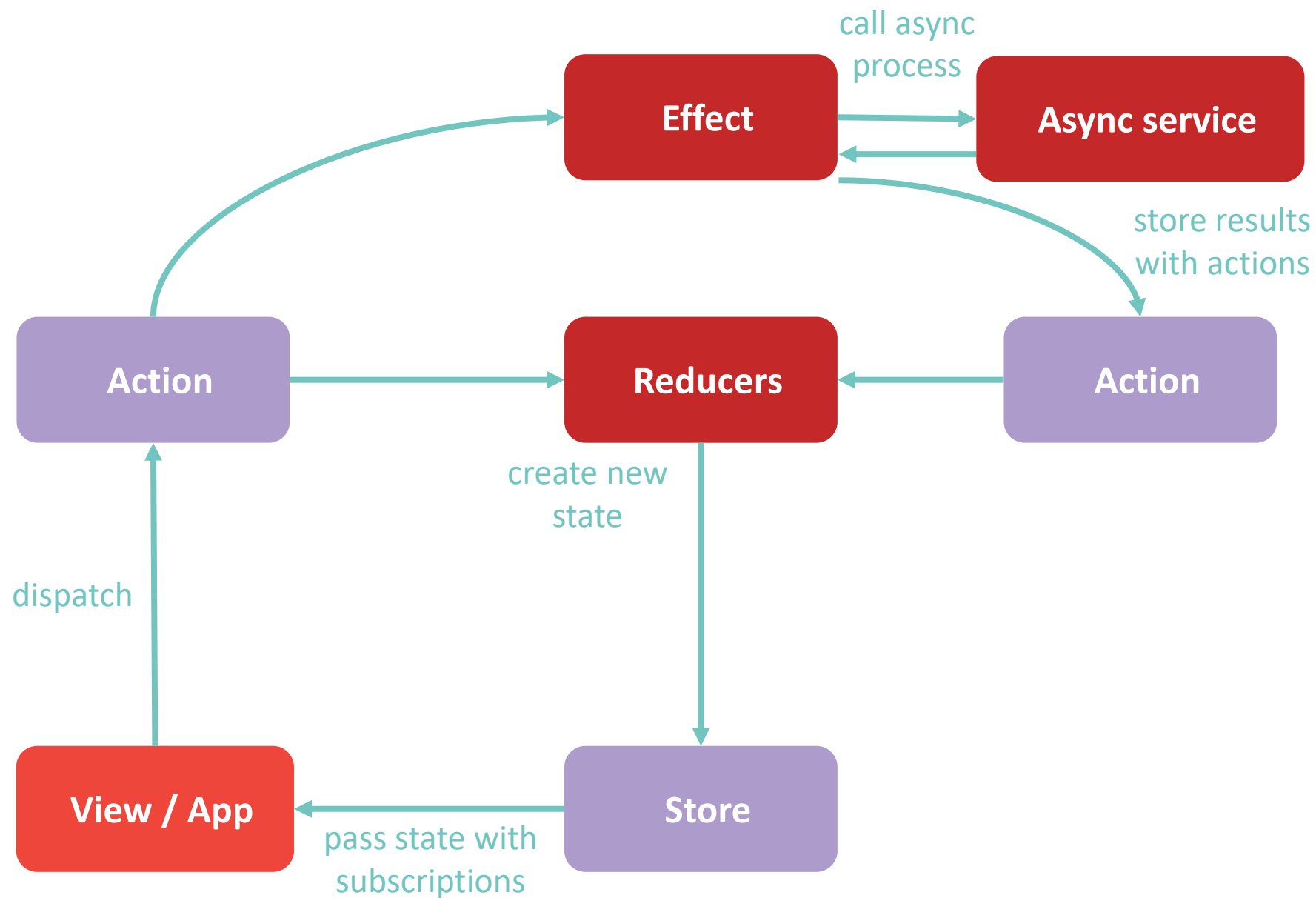
Retrieved with a side effect

Impacted by actions from other sources

https://www.youtube.com/watch?v=t3jx0EC-Y3c&ab_channel=ng-conf

What Should Not Go in the Store?

- Unshared state
- Angular form state
- Non serializable state



Effects

```
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { EMPTY } from 'rxjs';

constructor(private actions$: Actions, private loginService: LoginService) {}

loadLogin$ = createEffect(() => this.actions$.pipe(
  ofType('LOGIN'), // or `ofType(loginAction)` when created with createAction()
  mergeMap((action) => {
    return this.loginService.login(action.payload)
      .pipe(
        map(credentials => ({ type: 'LOGIN_SUCCESS', payload: credentials })),
        catchError(() => EMPTY)
      )
  })
);
```

The loadLogin\$ effect is listening for all dispatched actions through the Actions stream, but is only interested in the LOGIN event using the ofType operator. The stream of actions is then flattened and mapped into a new observable using the mergeMap operator. The LoginService#login() method returns an observable that maps the credentials to a new action on success, and currently returns an empty observable if an error occurs. The action is dispatched to the Store where it can be handled by reducers when a state change is needed. Its also important to handle errors when dealing with observable streams so that the effects continue running.

Complex code:

- Hard to debug
- Every action must be mapped to exactly 1 other action
- Your algorithm is spread in your code

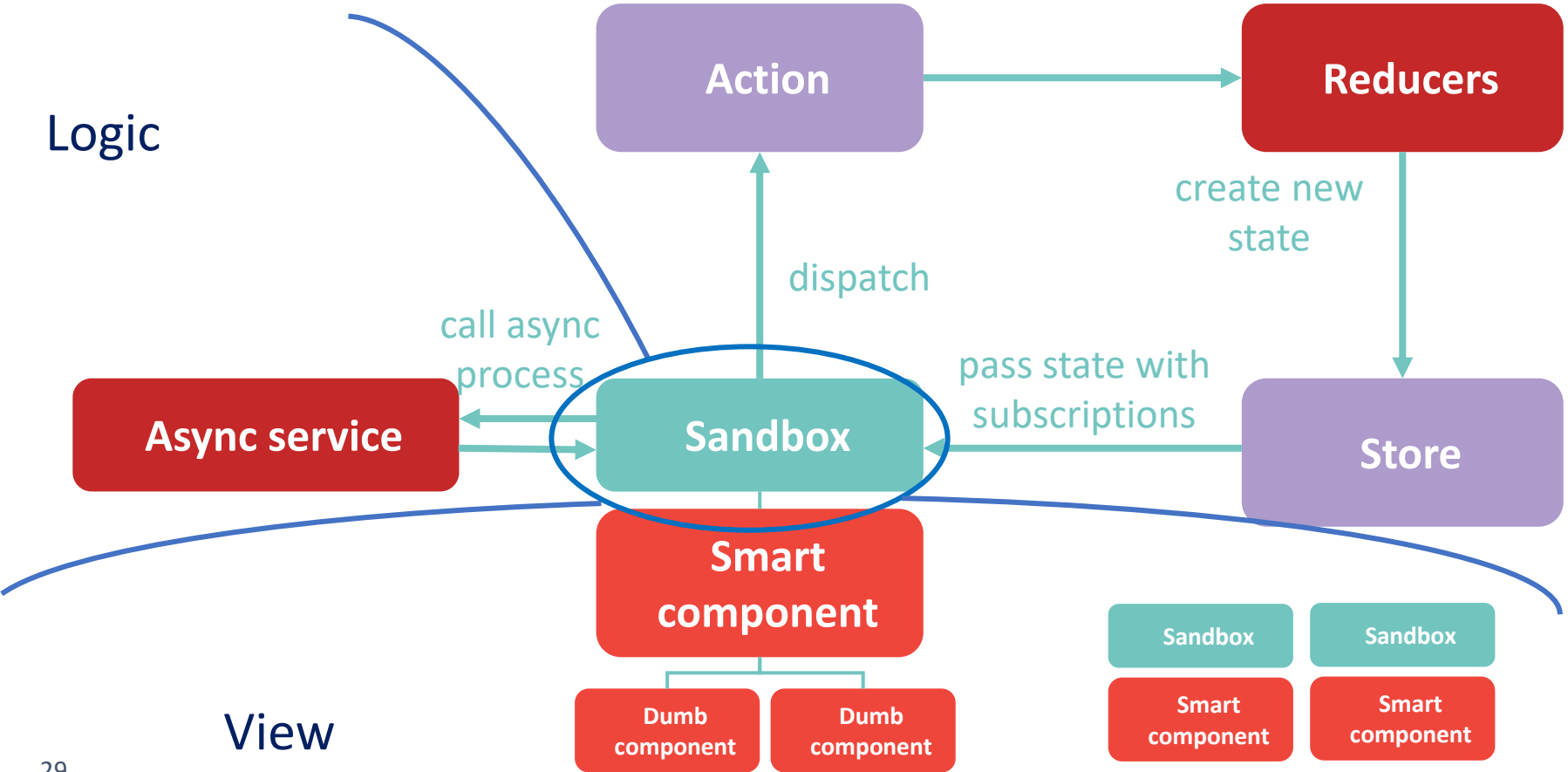
Sandboxes

- (Brecht Billet)
 - <https://blog.strongbrew.io/A-scalable-angular2-architecture/>
 - <https://blog.strongbrew.io/A-scalable-angular-architecture-part2/>
- 1. Don't let your components play with all the toys
- 2. Components should not know about the state management layer
- 3. HTTP services should not know about the state management layer

Very similar to NGRX Façade pattern:

<https://medium.com/@thomasburlesonIA/ngrx-facades-better-state-management-82a04b9a1e39>

State management



Responsibilities of the sandbox

- Sole interface for a smart component (container) to the outside world
- Only exposes what this container needs
- 1 sandbox for 1 container
- Exposes
 - State using selectors
 - Actions using functions
 - Routing to other pages using functions

NEXT

TIME FOR ACTION



GIT Repo:

<https://github.com/pmestrum/angular-ngrx-exercise/tree/v18/start>

or

<https://bit.ly/4eBKKm9>





we commit. we deliver.